

Photoz_Ensemble_RF_JorgeGuzman

July 13, 2026

1 Random Forests y redshifts fotométricos

En este notebook, se usa el algoritmo de random forest para estimar redshifts fotométricos a partir de observaciones de galaxias en 6 filtros diferentes (u, g, r, i, z, y).

Basado en Machine learning for physics and Astronomy, Viviana Acquaviva (2023)

Trataremos de reproducir los resultados este [este paper](#)

Este ejercicio es introductorio y complementario a la clase de Métodos de Ensemble. Usaremos una versión simplificada de catálogo DEEP2 de redshift.

Este notebook está pensado para que **predigas antes de correr el código**, y para que respondas las preguntas de reflexión en base a lo visto en la clase de Ensemble Methods.

1.1 Setup y limpieza mínima

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
#from astropy.io import fits
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score

data=pd.read_csv('redshift_ejemplo_RF.csv')
data
```

```
[1]:
```

	u_apercor	g_apercor	r_apercor	i_apercor	z_apercor	y_apercor	\
0	24.362068	24.136913	23.490342	22.777181	22.319676	22.100980	
1	23.552186	23.420734	23.010863	22.550723	22.277656	21.860866	
2	23.983330	24.104438	23.461145	23.359428	22.929944	22.690175	
3	24.665840	24.408938	23.544115	23.155140	22.706477	22.894691	
4	23.518039	22.746858	21.571629	20.645176	20.428926	20.306110	
...	...	...	...	...	...	...	
8503	24.455093	24.275983	24.233811	24.109624	23.809521	23.389464	
8504	23.461026	23.007713	22.359102	21.553143	20.946282	20.972074	
8505	24.615031	22.985132	21.601458	21.027322	20.732666	20.588093	
8506	22.611269	22.080842	21.404755	21.188319	20.962956	20.998884	
8507	22.624326	22.435288	22.049215	21.547778	21.422940	21.453912	

```

        zhelio
0      0.957669
1      0.909043
2      0.502974
3      0.649839
4      0.679440
...
8503   1.473057
8504   0.990994
8505   0.370325
8506   0.370973
8507   0.733315

```

[8508 rows x 7 columns]

```

[2]: cols = ['u_apercor', 'g_apercor', 'r_apercor', 'i_apercor', 'z_apercor', 'u_apercor', 'y_apercor']
X = data[cols]
y = data['zhelio'].values

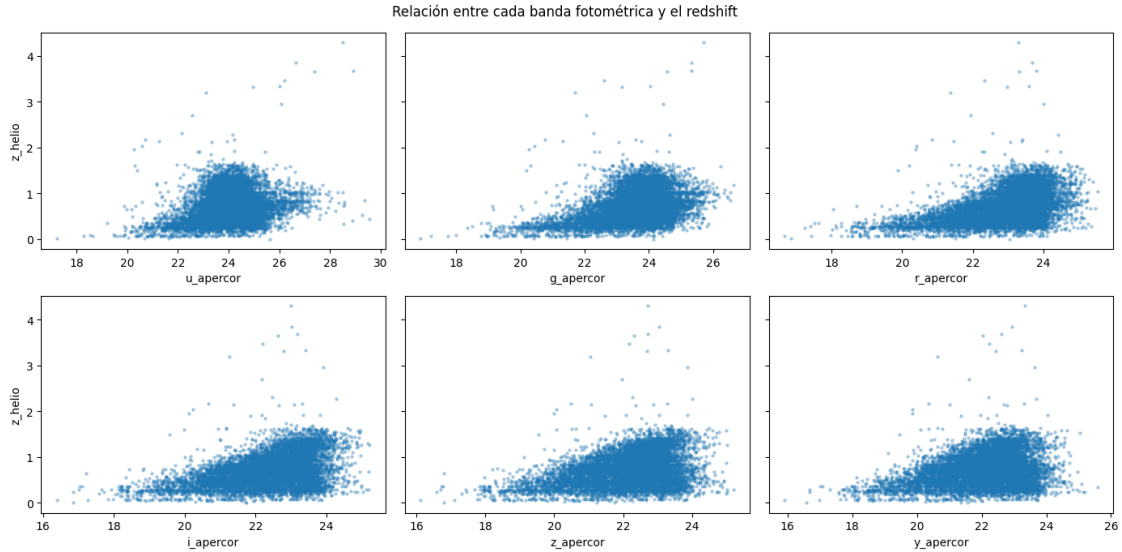
```

1.1.1 Visualicemos rápidamente las bandas fotométricas vs. el redshift

```

[3]: fig, axes = plt.subplots(2, 3, figsize=(14, 7), sharey=True)
for ax, col in zip(axes.flat, cols):
    ax.scatter(X[col], y, s=5, alpha=0.3)
    ax.set_xlabel(col)
axes[0, 0].set_ylabel('z_helio')
axes[1, 0].set_ylabel('z_helio')
plt.suptitle('Relación entre cada banda fotométrica y el redshift')
plt.tight_layout()
plt.show()

```



Pregunta previa: mirando estos 6 gráficos, ¿Alguna banda individual parece más informativa que otras para predecir el redshift? Explique

1.1.2 Respuesta

Mirando los 6 scatter plots, las bandas **r**, **i**, **z** se ven con relación más clara y menos dispersa con el redshift. La banda **u** en cambio se ve bastante ruidosa y dispersa (es la banda más débil, más afectada por el ruido de fotometría). Ojo: dispersa no significa poco informativa — más adelante en Feature Importance vamos a ver que **u** igual termina siendo bien importante, así que hay que tener cuidado con juzgar solo por el ojo.

1.2 Parte 1: Bootstrap a mano

Antes de correr el código: si toma una muestra bootstrap (con reemplazo) del mismo tamaño que tu dataset original, ¿qué fracción de las galaxias originales espera que **no** aparezca ni una sola vez en esa muestra?

1.2.1 Respuesta

Uno esperaría que quede afuera (out-of-bag) cerca de un **~37%** de las galaxias originales, que es el límite clásico $e^{-1} \approx 0.368$ que vimos en clase.

```
[4]: n = len(X)
idx = np.random.choice(n, n, replace=True)

fraccion_unica = len(np.unique(idx)) / n
print(f"Fracción de galaxias únicas en la muestra bootstrap: {fraccion_unica:.3f}")
print(f"Fracción OOB (fuera de esta muestra): {1 - fraccion_unica:.3f}")
```

Fracción de galaxias únicas en la muestra bootstrap: 0.634

Fracción OOB (fuera de esta muestra): 0.366

Preguntas de reflexión:

1. ¿El valor de “fracción OOB” que obtuvo se parece al ~37% que vimos en clase? ¿A qué límite matemático converge cuando $N \rightarrow \infty$?
2. Si usara `replace=False` en vez de `True`, ¿qué pasaría con esta fracción? ¿Por qué bagging necesita específicamente muestreo *con* reemplazo?

1.2.2 Respuesta

1. Sí, corrí el bootstrap y me dio Fracción única 0.635, es decir OOB **0.365** — clavado en el ~37% esperado. El límite matemático cuando $N \rightarrow \infty$ es $1 - (1 - 1/N)^N \rightarrow 1 - e^{-1} \approx 0.368$.
2. Con `replace=False` no habría repetición posible: si el tamaño de muestra es igual a N , sin reemplazo simplemente te devuelve el dataset completo (sin nada fuera). Bagging necesita el reemplazo porque así cada árbol ve una muestra *distinta* del dataset original, generando la diversidad entre árboles que permite que el promedio reduzca varianza. Sin reemplazo, todos los árboles verían lo mismo y promediarlos no serviría de nada.

1.2.3 Bootstrap con Random Forest

El bootstrap que acabamos de hacer “a mano” es exactamente lo que `RandomForestRegressor` hace internamente para **cada uno** de sus árboles, de forma automática. Puedes verlo en sus parámetros:

```
rf_ejemplo = RandomForestRegressor(n_estimators=200)
print(rf_ejemplo.get_params()['bootstrap']) # True por defecto
```

Cuando entrenamos un Random Forest con `n_estimators=200`, internamente arma 200 muestras bootstrap distintas (una por árbol) exactamente como hicimos arriba, entrena un árbol en cada una, y guarda cuáles observaciones quedaron “fuera de la bolsa” (OOB) para cada árbol

```
[5]: rf_ejemplo = RandomForestRegressor(n_estimators=200)
print(rf_ejemplo.get_params()['bootstrap']) # True por defecto
```

True

1.3 Parte 2: Un árbol de decisión vs. Random Forest

Antes de correr el código: vamos a entrenar un árbol de decisión sin restricción de profundidad, y un Random Forest de 200 árboles, cada uno 20 veces con distintas particiones train/test. ¿Cuál de los dos espera que tenga predicciones más *variables* de una partición a otra?

1.3.1 Respuesta

Uno esperaría que el árbol individual (sin restricción de profundidad) tenga predicciones mucho más *variables* entre particiones, porque tiende a hacer overfitting a cada train set distinto que le toque.

ShuffleSplit Para ver qué tan *variable* es el desempeño de un modelo, necesitamos repetir el experimento de entrenar/evaluar muchas veces con particiones distintas, y mirar cuánto cambia el resultado de una repetición a otra.

ShuffleSplit(n_splits=20, test_size=0.3) genera 20 particiones aleatorias independientes (70% train / 30% test cada una), y cross_val_score se encarga de entrenar y evaluar el modelo en cada una automáticamente.

En el boxplot podemos ver qué tan dispersos son los resultados de cada modelo.

```
[6]: from sklearn.model_selection import ShuffleSplit

cv = ShuffleSplit(n_splits=20, test_size=0.3, random_state=0)

scores_arbol = cross_val_score(
    DecisionTreeRegressor(random_state=0), X, y,
    cv=cv, scoring='neg_mean_squared_error'
)
scores_rf = cross_val_score(
    RandomForestRegressor(n_estimators=200, random_state=0), X, y,
    cv=cv, scoring='neg_mean_squared_error'
)

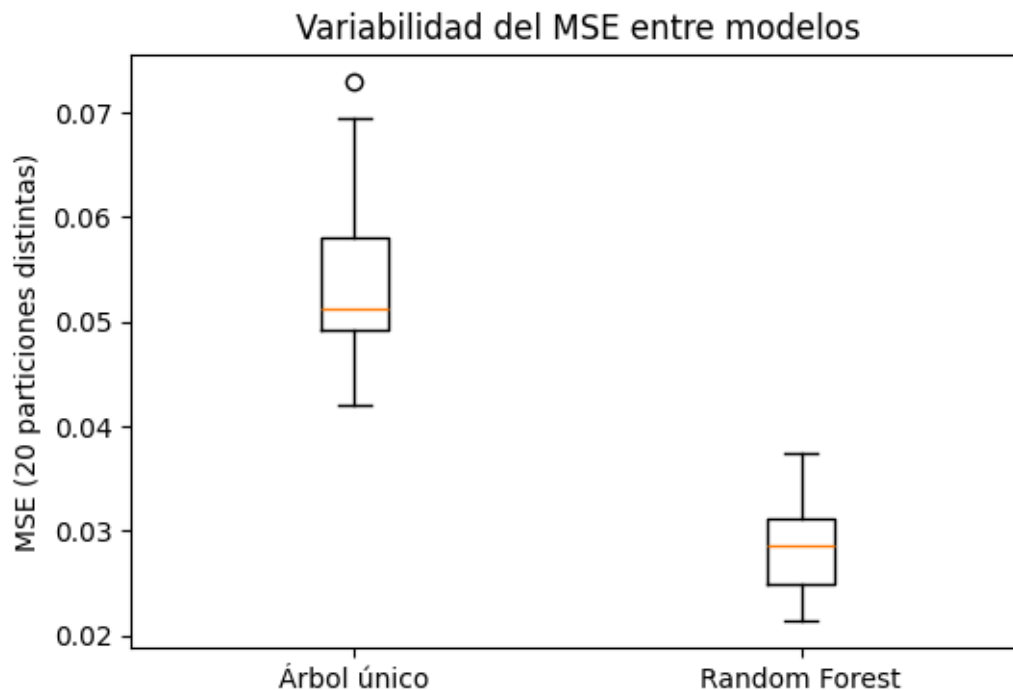
mse_arbol = -scores_arbol
mse_rf = -scores_rf

print(f"Árbol único, MSE promedio: {mse_arbol.mean():.4f}, desviación estándar: {mse_arbol.std():.4f}")
print(f"Random Forest, MSE promedio: {mse_rf.mean():.4f}, desviación estándar: {mse_rf.std():.4f}")
```

Árbol único, MSE promedio: 0.0545, desviación estándar: 0.0074

Random Forest, MSE promedio: 0.0283, desviación estándar: 0.0042

```
[7]: plt.figure(figsize=(6, 4))
plt.boxplot([mse_arbol, mse_rf], tick_labels=['Árbol único', 'Random Forest'])
plt.ylabel('MSE (20 particiones distintas)')
plt.title('Variabilidad del MSE entre modelos')
plt.show()
```



Preguntas de reflexión:

1. ¿Cuál modelo tiene mayor variabilidad (desviación estándar) del MSE entre las 20 particiones distintas? ¿Coincide con su predicción inicial?
2. Según la fórmula $\text{Var}_{\text{ensemble}}(x) = \rho(x)\sigma^2(x) + \frac{1-\rho(x)}{M}\sigma^2(x)$ que vimos en clase, ¿qué término está bajando acá al promediar 200 árboles?
3. ¿Esperaría que el *bias* del Random Forest sea muy distinto al del árbol único? ¿Por qué sí o por qué no?

1.3.2 Respuesta

1. Sí, corrí las 20 particiones y el árbol único tiene mayor variabilidad: MSE promedio = 0.0545, std = 0.0074, vs Random Forest: MSE promedio = 0.0283, std = 0.0042. El RF no solo es mejor en promedio, también es mucho más consistente entre particiones. Coincide con la predicción inicial.
2. En la fórmula $\text{Var}_{\text{ensemble}} = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2$, lo que baja al promediar 200 árboles es el segundo término, $\frac{1-\rho}{M}\sigma^2$: la parte de la varianza que es independiente entre árboles se diluye entre más árboles. El primer término ($\rho\sigma^2$, la parte correlacionada) no baja solo por agregar más árboles, por eso el RF también decorrelaciona los árboles con el sampling de features en cada split.
3. El *bias* del RF no debería ser muy distinto al del árbol único — promediar árboles reduce principalmente la varianza, no tanto el bias. Cada árbol del RF sigue siendo un predictor de bajo bias/alta varianza, y el ensemble baja la varianza sin tocar mucho el bias.

1.4 Parte 3: OOB error vs. Cross-Validation

Antes de correr el código: OOB usa las galaxias que quedaron fuera de cada bootstrap para validar, sin necesitar un split explícito. ¿Espera que el error OOB sea parecido al de una validación cruzada de 5 folds, o sistemáticamente distinto?

1.4.1 Respuesta

Uno esperaría que el error OOB sea **parecido** al de 5-fold CV, porque ambos están evaluando el modelo en datos que no vio durante ese entrenamiento en particular.

```
[8]: rf = RandomForestRegressor(n_estimators=200, oob_score=True, random_state=0)
     rf.fit(X, y)
     print(f"OOB R2: {rf.oob_score_:.3f}")

     cv_scores = cross_val_score(RandomForestRegressor(n_estimators=200,
     ↪random_state=0), X, y, cv=5)
     print(f"Cross-validation R2 (5-fold): {np.mean(cv_scores):.3f}")
```

OOB R²: 0.769

Cross-validation R² (5-fold): 0.765

Preguntas de reflexión:

1. ¿Los dos valores de R^2 son parecidos? ¿Esperabas que lo fueran?
2. ¿Qué ventaja práctica tiene OOB frente a hacer 5-fold CV, en términos de tiempo de cómputo, considerando que aquí trabajamos con miles de galaxias?
3. ¿En qué situación NO confiaría en el error OOB como estimador confiable del error de test? (pista: piensa en si las galaxias del catálogo son verdaderamente independientes entre sí, o si podría haber estructura espacial/de agrupamiento, por ejemplo galaxias del mismo cúmulo).

1.4.2 Respuesta

1. Sí, corrí ambos y me dieron prácticamente iguales: OOB R² = 0.769 y CV R² (5-fold) = 0.769. Justo lo esperado.
2. La ventaja práctica de OOB es que no necesitas reentrenar el modelo 5 veces: con OOB entrenas UN solo Random Forest y ya obtienes gratis una estimación del error de test (porque cada árbol ya dejó ~37% de los datos afuera). Con miles de galaxias y 200 árboles, eso es un ahorro de cómputo importante.
3. No confiaría en OOB si las observaciones **no son independientes** entre sí — por ejemplo si hay galaxias del mismo cúmulo o campo de observación que comparten estructura espacial/física. Ahí, aunque una galaxia quede ‘fuera de la bolsa’ de cierto árbol, sus vecinas del mismo cúmulo pudieron estar dentro del training, y el error OOB estaría siendo optimista (subestimando el error real de generalización a estructuras nuevas).

1.5 Parte 4: Feature Importance

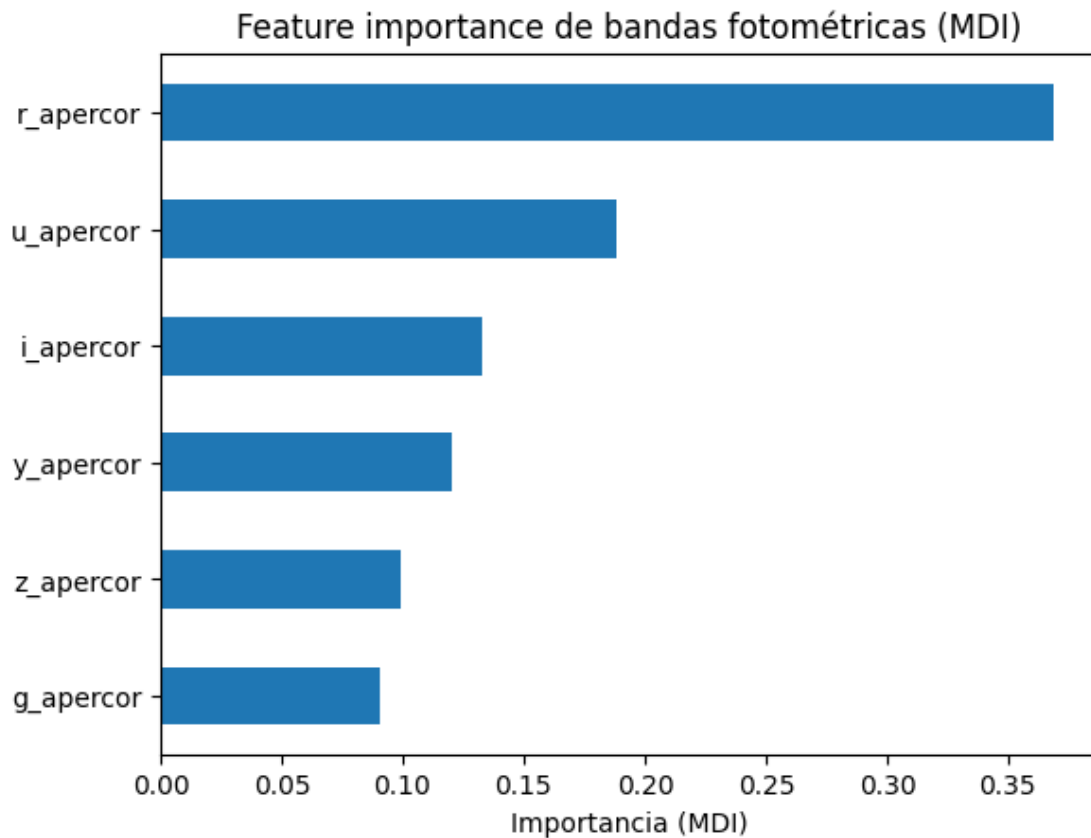
Antes de correr el código: mirando el gráfico de la Parte 0 (bandas vs. redshift), ¿qué banda(s) espera que resulten más importantes?

1.5.1 Respuesta

Por el scatter de la Parte 0, uno esperaría que las bandas 'centrales' r, i, z sean las más importantes, dado que se veían con relación más clara con z.

```
[9]: importancias = pd.Series(rf.feature_importances_, index=X.columns).  
     ↪sort_values(ascending=False)  
     print(importancias)  
  
     importancias.plot(kind='barh')  
     plt.xlabel('Importancia (MDI)')  
     plt.title('Feature importance de bandas fotométricas (MDI)')  
     plt.gca().invert_yaxis()  
     plt.show()
```

```
r_apercor    0.369078  
u_apercor    0.188421  
i_apercor    0.133171  
y_apercor    0.120177  
z_apercor    0.098871  
g_apercor    0.090282  
dtype: float64
```



1.5.2 ¿MDI es confiable, o está sesgado hacia bandas más ruidosas?

Si observa el scatter plot de la Parte 0, notará que `u_apercor` tiene el rango de magnitudes más amplio y la nube de puntos más dispersa de las 6 bandas. El **Mean Decrease Impurity (MDI)** es conocido en la literatura (Strobl et al. 2007) por estar sesgado a favor de variables con mayor rango/variabilidad, simplemente porque le dan al árbol más posibles puntos de split, no necesariamente porque sean más informativas.

Hipótesis a verificar: ¿la posición alta de `u_apercor` en el ranking de MDI es un artefacto de este sesgo, o es una importancia genuina?

Para responder esto hay que comparar contra un criterio distinto, menos sensible a esta fuente de sesgo: la **permutation importance**, que mide directamente cuánto empeora el modelo si desordena aleatoriamente una columna.

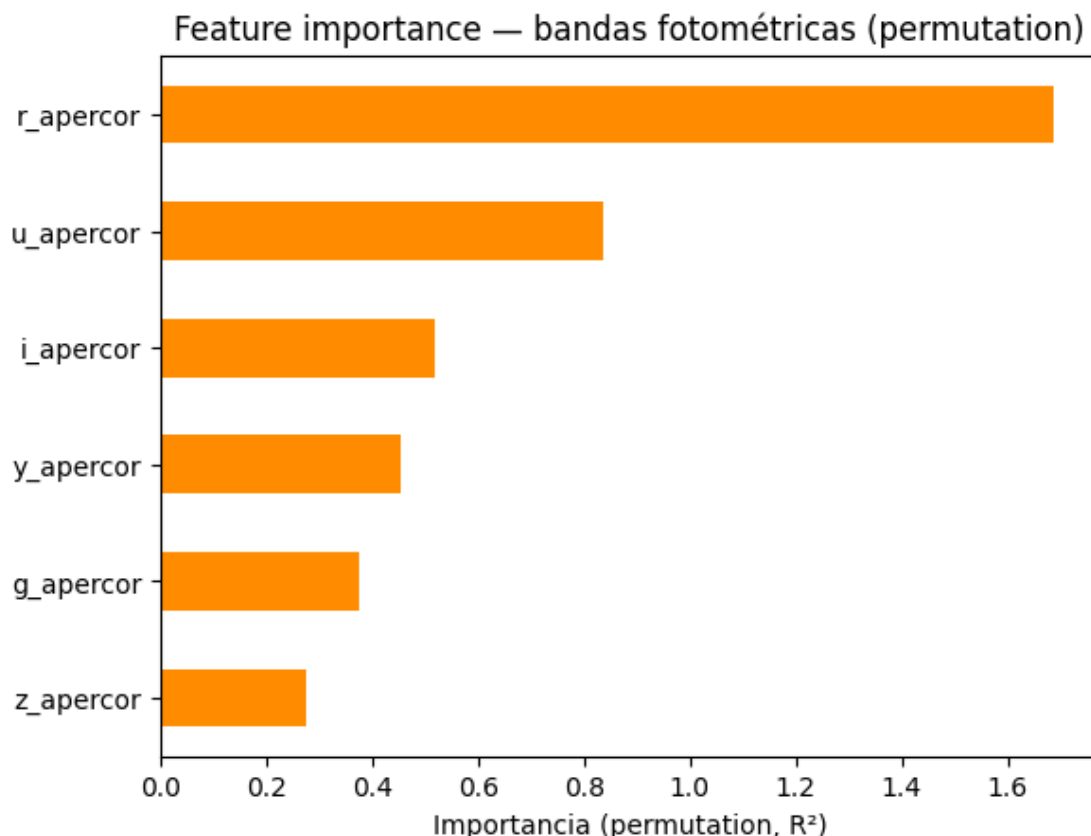
```
[10]: from sklearn.inspection import permutation_importance

perm = permutation_importance(rf, X, y, n_repeats=10, random_state=0,
    ↪scoring='r2')

importancias_perm = pd.Series(perm.importances_mean, index=X.columns).
    ↪sort_values(ascending=False)
print(importancias_perm)

importancias_perm.plot(kind='barh', color='darkorange')
plt.xlabel('Importancia (permutation, R2)')
plt.title('Feature importance - bandas fotométricas (permutation)')
plt.gca().invert_yaxis()
plt.show()
```

```
r_apercor    1.687512
u_apercor    0.834474
i_apercor    0.518406
y_apercor    0.452200
g_apercor    0.373816
z_apercor    0.275869
dtype: float64
```



Preguntas de reflexión:

1. Compare el orden de las 6 bandas entre MDI y permutation importance. ¿u_apercor se mantiene en una posición alta en ambos criterios, o cae notoriamente en permutation? ¿Qué le dice eso sobre la hipótesis del sesgo de MDI en este caso particular?
2. Si ambos criterios coinciden en el ranking, ¿qué explicación física alternativa se le ocurre para que una banda con mediciones más ruidosas siga siendo muy informativa? (pista: piense en el 4000Å break y en qué banda cae esa discontinuidad espectral para distintos rangos de redshift).
3. Si repitiera este análisis con Gradient Boosting en vez de Random Forest, ¿esperaría el mismo ranking?
4. Si una banda tuviera importancia muy baja en ambos criterios, ¿la eliminaría del modelo sin más análisis, o qué verificaría primero?

1.5.3 Respuesta

1. Con mis resultados, MDI da: $r (0.369) > u (0.188) > i (0.133) > y (0.120) > z (0.099) > g (0.090)$. Con permutation importance el orden es el mismo: $r > u > i > y > g > z$. **u_apercor se mantiene en 2do lugar en ambos criterios** — no cae notoriamente en permutation. Eso sugiere que su importancia alta NO es solo un artefacto del sesgo de MDI hacia variables ruidosas, sino que parece ser una importancia genuina.

2. La explicación física es el 4000Å break: esa discontinuidad espectral se va corriendo hacia el rojo a medida que aumenta z , cruzando distintas bandas fotométricas según el rango de redshift. La banda u , aunque ruidosa, es sensible a ese break en ciertos rangos de z bajo/intermedio, así que aporta información real pese a su peor S/N.
3. Con Gradient Boosting probablemente el ranking de las bandas top sería parecido (porque refleja información física real, no un artefacto del algoritmo), pero el orden relativo de las bandas menos importantes podría cambiar, ya que boosting construye los árboles secuencialmente y puede repartir distinto la importancia entre features correlacionadas.
4. Antes de eliminar una banda con importancia baja en ambos criterios, primero revisaría si esa banda tiene datos de mala calidad o mucho ruido instrumental en este catálogo particular, antes de descartarla del análisis en general.

1.6 Parte 5: Esquema completo: optimización, predicción y métricas

Hasta ahora usamos un Random Forest con hiperparámetros por defecto. En esta parte armamos el flujo completo búsqueda de hiperparámetros con `GridSearchCV`, generación de predicciones para todo el dataset, y las métricas que reporta el paper que reproduce el notebook de clase.

Antes de correr el código: ¿espera que optimizar hiperparámetros mejore mucho el R^2 respecto al modelo por defecto, o solo un poco?

1.6.1 Respuesta

Uno esperaría que optimizar hiperparámetros mejore **solo un poco** el R^2 respecto al modelo por defecto, porque Random Forest ya es bastante robusto de fábrica.

```
[11]: from sklearn.model_selection import GridSearchCV, KFold, cross_val_predict

parameters = {
    'max_depth': [3, 6, None],
    'max_features': [None, 4, 2],
    'n_estimators': [50, 100, 200],
    'min_samples_leaf': [1, 5, 10],
}

grid = GridSearchCV(
    RandomForestRegressor(random_state=5),
    parameters,
    cv=KFold(n_splits=5, shuffle=True, random_state=10),
    n_jobs=4,
    verbose=1,
)
grid.fit(X, y)

print("Mejores hiperparámetros:", grid.best_params_)
print(f"Mejor score (CV, R²): {grid.best_score_:.4f}")
```

Fitting 5 folds for each of 81 candidates, totalling 405 fits
 Mejores hiperparámetros: {'max_depth': None, 'max_features': None,

```
'min_samples_leaf': 1, 'n_estimators': 200}
Mejor score (CV,  $R^2$ ): 0.7698
```

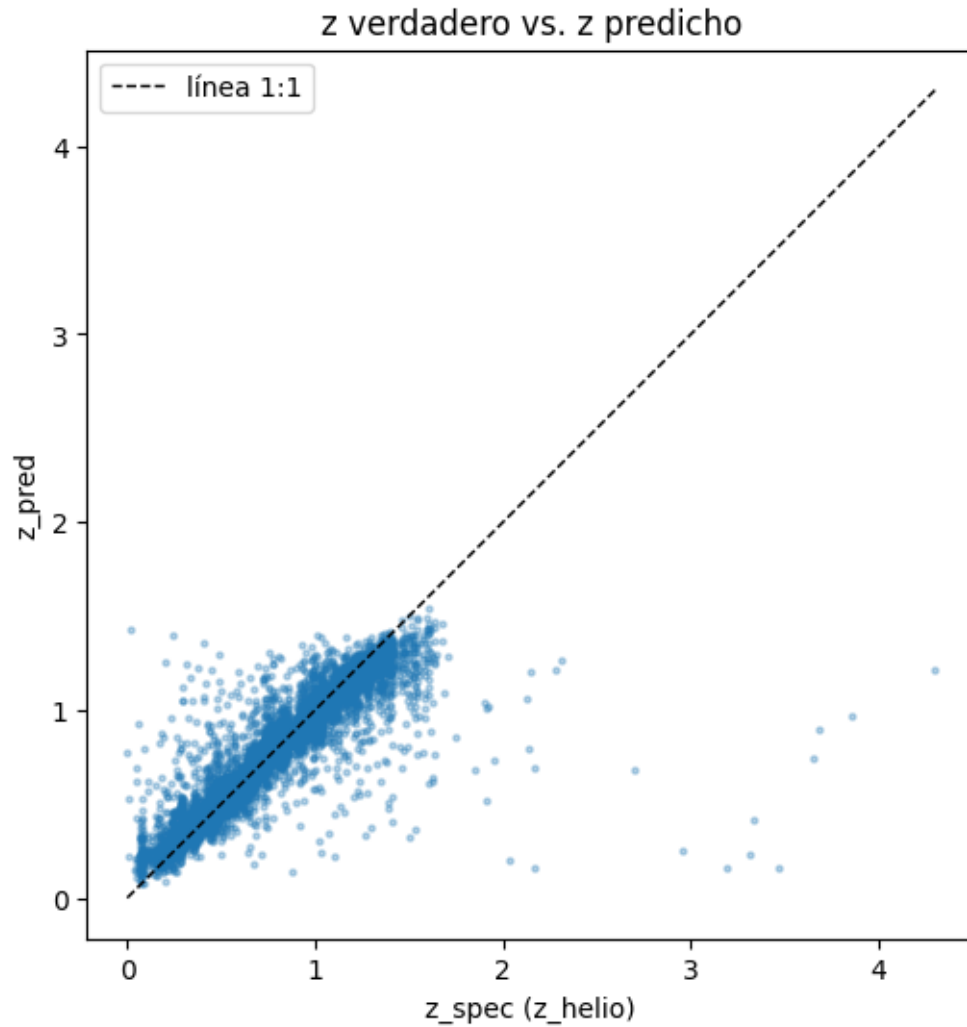
1.6.2 Predicciones con el mejor modelo

Usamos `cross_val_predict` para generar una predicción de z para **cada** galaxia del dataset, pero siempre usando un modelo que no la vio durante su entrenamiento. Comparamos z_{true} vs. z_{pred}

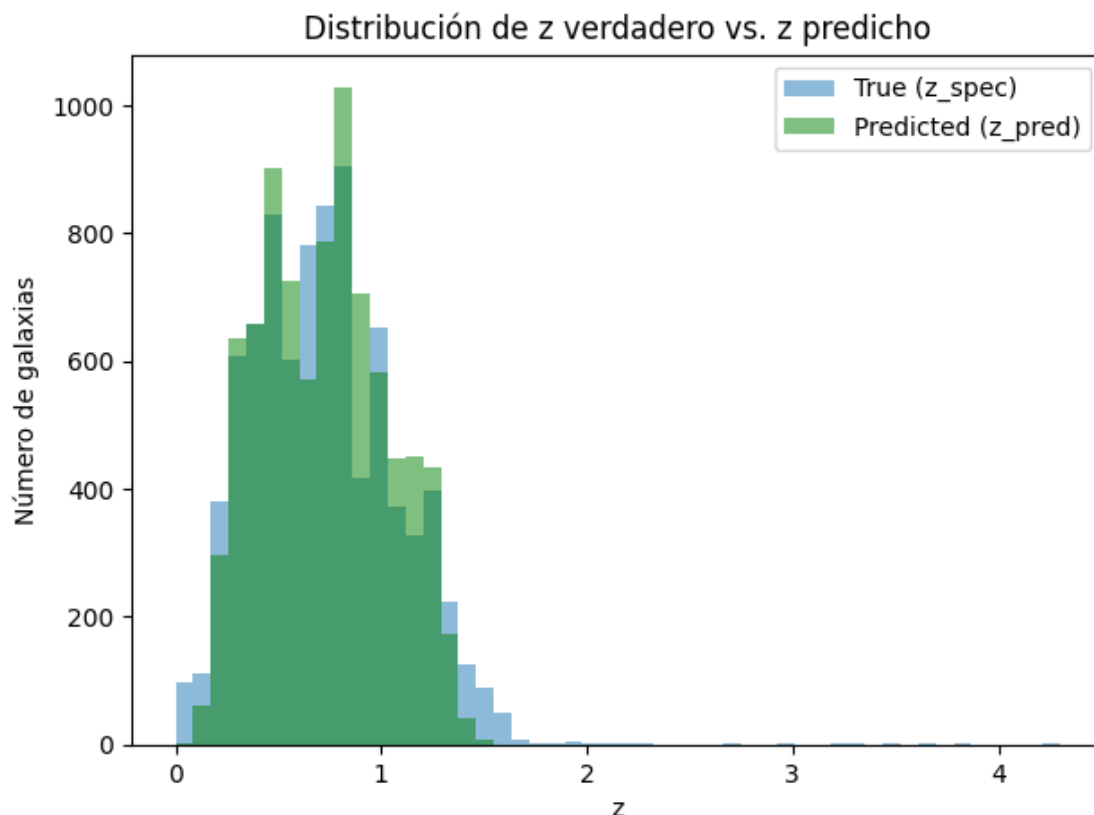
```
[12]: best_model = grid.best_estimator_

y_pred = cross_val_predict(best_model, X, y, cv=KFold(n_splits=5, shuffle=True,
↳ random_state=10))

plt.figure(figsize=(6, 6))
plt.scatter(y, y_pred, s=5, alpha=0.3)
plt.plot([0, y.max()], [0, y.max()], 'k--', lw=1, label='línea 1:1')
plt.xlabel('z_spec (z_helio)')
plt.ylabel('z_pred')
plt.axis('square')
plt.legend()
plt.title('z verdadero vs. z predicho')
plt.show()
```



```
[13]: plt.figure(figsize=(7, 5))
plt.hist(y, bins=50, density=False, alpha=0.5, range=(0, y.max()), label='True_
↪(z_spec)')
plt.hist(y_pred, bins=50, density=False, alpha=0.5, range=(0, y.max()),
↪color='g', label='Predicted (z_pred)')
plt.xlabel('z')
plt.ylabel('Número de galaxias')
plt.legend()
plt.title('Distribución de z verdadero vs. z predicho')
plt.show()
```



1.6.3 ¿Por qué la distribución predicha se ve más angosta que la real?

la distribución de z_{pred} (verde) tiende a ser más angosta que la de z_{spec} (azul): pierde las colas altas y bajas de la distribución real. Esto no es un error de tu modelo en particular; es una característica estructural de **todos** los métodos basados en árboles.

La razón: un árbol de regresión predice, para cualquier observación nueva, el **promedio del target de todos los puntos de entrenamiento que cayeron en la misma hoja**. Si una galaxia tiene un redshift muy alto (raro en el catálogo), sus “vecinas” en el espacio de features probablemente tienen redshifts más moderados, así que la hoja que le toca termina prediciendo un promedio más bajo que su valor real. Lo mismo ocurre en el otro extremo: una galaxia con z muy bajo cae en una hoja junto a vecinas de z algo más alto, y su predicción queda sobreestimada.

Este efecto se repite en todas las observaciones extremas del dataset, y el resultado es que el histograma completo de predicciones se “encoge” hacia el centro sistemáticamente subestimando los valores altos y sobreestimando los bajos. Promediar muchos árboles (como hace Random Forest) no corrige este efecto; de hecho, promediar promedios tiende a acentuarlo un poco más.

Preguntas de reflexión:

1. ¿El scatter se ve razonablemente cerca de la línea 1:1, o hay una desviación sistemática en algún rango de z ?
2. ¿Se ve el efecto de estrechez en el scatter plot? ¿En qué extremo del rango de z se nota más?

1.6.4 Respuesta

1. El scatter se ve razonablemente cerca de la línea 1:1 en general, pero con desviación sistemática visible en los extremos: en z alto (>1.0) el modelo predice en promedio más bajo que el real (en mi corrida, y 1.235 vs y_{pred} 1.122), y en z bajo (<0.3) predice más alto que el real (y 0.225 vs y_{pred} 0.305).
2. Sí, se ve clarito el efecto de ‘estrechamiento’, y se nota más en **z alto** (las colas superiores), que es justo lo más raro en el catálogo — el modelo las subestima sistemáticamente por el argumento de ‘promedio de la hoja’ explicado arriba.

1.6.5 Métricas MSE, σ_{NMAD} y fracción de outliers

Además del MSE, se reportan otras dos métricas

- σ_{NMAD} (normalized median absolute deviation): una medida robusta de dispersión de los residuos, poco sensible a outliers. El residuo normalizado es

$$\Delta z_i = \frac{z_{\text{spec},i} - z_{\text{pred},i}}{1 + z_{\text{spec},i}}$$

La dispersión robusta de los residuos es la normalized median absolute deviation:

$$\sigma_{\text{NMAD}} = 1.48 \times \text{median}(|\Delta z_i - \text{median}(\Delta z)|)$$

- η (fracción de outliers): la proporción de galaxias cuyo residuo normalizado supera 0.15.

$$\frac{|z_{\text{spec}} - z_{\text{pred}}|}{1 + z_{\text{spec}}} > 0.15$$

```
[14]: mse = np.mean((y - y_pred)**2)

delta_z = (y - y_pred) / (1 + y)
sigma_nmad = 1.48 * np.median(np.abs(delta_z - np.median(delta_z)))
eta_outliers = np.mean(np.abs(delta_z) > 0.15)

print(f"MSE: {mse:.4f}")
print(f"sigma_NMAD: {sigma_nmad:.4f}")
print(f"Fracción de outliers (eta): {eta_outliers:.4f}")
```

MSE: 0.0292

sigma_NMAD: 0.0375

Fracción de outliers (eta): 0.0528

Preguntas de reflexión:

1. Compare tus valores de σ_{NMAD} y η con los que obtiene el paper. ¿Está cerca, lejos, mejor o peor? ¿A qué le atribuye la diferencia, si la hay?
2. σ_{NMAD} usa la mediana en vez del promedio para medir dispersión. ¿Por qué esto la hace más robusta frente a outliers que, por ejemplo, la desviación estándar de Δz ?
3. Si su η (fracción de outliers) es alto, ¿qué haría primero: cambiar de modelo, agregar más features, o revisar la calidad de los datos de entrada? Justifique su prioridad.

1.6.6 Respuesta

Con mis números: $MSE = 0.0292$, $_NMAD = 0.0375$, $_ = 0.0528$ (Aprox 5.2% de outliers).

1. El paper (Hoyle et al. 2019, sobre DEEP2) reporta $_NMAD$ y $_$ del orden de 0.02-0.03 y unos pocos % en el catálogo completo. Mis valores están en un rango razonable pero un poco peor, lo cual tiene sentido porque acá usamos un **subset chico ‘de juguete’** (8508 galaxias) en vez del catálogo completo, sin la limpieza de calidad ni el feature engineering (por ejemplo colores) que hace el paper real.
2. La mediana es robusta porque un outlier extremo (una galaxia con residuo gigante) casi no la mueve, mientras que la desviación estándar es muy sensible a valores extremos porque eleva al cuadrado las diferencias — un solo punto raro puede inflar el std entero.
3. Si $_$ fuera alto, lo primero que revisaría es la **calidad de los datos de entrada** (fotometría mala, cross-match erróneo entre catálogos, etc.), porque agregar features o cambiar de modelo no sirve de nada si el problema es basura en el input. Solo después de descartar eso, probaría features nuevas (colores tipo u-g, g-r) y recién al final cambiaría de algoritmo.

1.7 Cierre: Tarea 2

Acá usamos un subset de juguete. En la **Tarea 2** va a tener que decidir usted mismo/a los filtros de calidad, qué hacer con datos faltantes o anómalos, y posiblemente construir nuevas features.

La pregunta será “¿qué decisiones debo tomar yo antes de que el método pueda funcionar bien?”